

Esercizio gRPC in Java — Servizio di monitoraggio sensori

1. Obiettivo

Realizzare un sistema cliente/servente basato su **gRPC** e **Protocol Buffers** che raccolga letture da sensori ambientali. L'esercizio esercita il percorso completo visto a lezione: definizione del contratto in un file `.proto`, compilazione con `protoc` e con il plugin `protoc-gen-grpc-java`, implementazione del servente per ereditarietà dalla classe `*ImplBase` generata, e invocazione dal cliente con i tre tipi di stub (bloccante, asincrono, future).

2. Dominio applicativo

Un *SensorHub* mantiene in memoria le letture prodotte da un insieme di sensori. Ogni lettura è composta da identificativo del sensore, tipo di grandezza misurata, valore e timestamp. Nessun database: una mappa concorrente è sufficiente.

3. Specifica del contratto (`sensor.proto`)

Completa lo scheletro dove indicato dai commenti. Mantieni `java_multiple_files = true` per evitare la classe contenitore esterna.

```
syntax = "proto3";

option java_multiple_files = true;
option java_package = "it.unina.acp.sensor";
option java_outer_classname = "SensorProto";

package sensor;

message Reading {
    string sensor_id = 1;
    string type = 2; // "temperature", "humidity", "pressure"
    double value = 3;
    int64 timestamp = 4; // epoch millis
}

message Ack { bool accepted = 1; string message = 2; }
message SensorQuery { string sensor_id = 1; int32 max_results = 2; }
message Stats { string sensor_id = 1; int32 count = 2;
                double average = 3; double max = 4; }

service SensorHub {
    // 3.1 – unaria: registra una singola lettura e restituisce Ack
    rpc RegisterReading(Reading) returns (Ack);

    // 3.2 – server streaming: COMPLETA LA FIRMA.
    // restituisce fino a max_results letture del sensore richiesto
    rpc StreamReadings(...) returns (...);

    // 3.3 – unaria, ma computazionalmente lenta: calcola le statistiche
    // di un sensore. Simula il costo con un Thread.sleep(2000).
    rpc ComputeStats(SensorQuery) returns (Stats);
}
```

4. Requisiti funzionali

4.1 Servente

- Avvio con `ServerBuilder.forPort(50051)`, registrazione del servizio con `addService()`, poi `start()` e `awaitTermination()`. Stampa in console la porta effettiva con `server.getPort()`: fallo girare anche con `forPort(0)` e osserva cosa cambia.
- Implementa il servizio estendendo `SensorHubGrpc.SensorHubImplBase` e sovrascrivendo i metodi. Ricorda che i metodi sono `void`: la risposta passa dallo `StreamObserver` ricevuto come parametro, mai da un `return`.
- `RegisterReading`: valida la lettura (`sensor_id` non vuoto, `type` fra i tre ammessi, `timestamp > 0`). Se valida, la memorizza e risponde con `accepted = true`; altrimenti chiudi la chiamata con `responseObserver.onError(Status.INVALID_ARGUMENT.withDescription(...).asRuntimeException())`.
- `StreamReadings`: emette una alla volta le letture memorizzate per il sensore, chiudendo con `onCompleted()`. Se il sensore non esiste, usa `Status.NOT_FOUND`. Inserisci un ritardo di ~300 ms fra un `onNext()` e il successivo, così lo streaming è osservabile.
- `ComputeStats`: un unico `onNext()` seguito da `onCompleted()`, con `Thread.sleep(2000)` prima della risposta per rendere visibile la differenza fra stub bloccante e non bloccante.
- Lo stato condiviso deve essere thread-safe: gRPC serve le richieste con un pool di thread, quindi più RPC possono toccare la mappa in parallelo. Prova anche ad avviare il server con un `Executors.newFixedThreadPool(2)` passato a `ServerBuilder.executor()` e osserva il comportamento con tre clienti simultanei.

4.2 Cliente

- Crea il canale con `Grpc.newChannelBuilder("localhost:50051", InsecureChannelCredentials.create()).build()` e chiudilo nel blocco `finally` con `shutdownNow().awaitTermination(5, TimeUnit.SECONDS)`.
- Segui la convenzione dell'esempio Hello World: la classe cliente riceve nel costruttore un `Channel`, non un `ManagedChannel`. Sai motivare la scelta?
- Stub bloccante** per `RegisterReading`: registra 5 letture valide e 1 non valida, intercettando la `StatusRuntimeException` e stampando `e.getStatus().getStatusCode()`.

- **Stub bloccante** anche per `StreamReadings`: il metodo generato restituisce un `Iterator<Reading>`; stampa le letture man mano che arrivano.
- **Stub asincrono** (`newStub`) per `ComputeStats`: passa uno `StreamObserver<Stats>` anonimo con i tre metodi `onNext`, `onError`, `onCompleted`, e stampa qualcosa subito dopo la chiamata per verificare che il main non si sia bloccato. Sincronizza la terminazione con un `CountDownLatch`.
- **Facoltativo** — ripeti `ComputeStats` con lo **stub future** (`newFutureStub`): la chiamata torna subito un `ListenableFuture<Stats>` e sei tu a decidere quando bloccarti con `get()`.

5. Compilazione del contratto

Come visto a lezione, servono **due invocazioni distinte** dello stesso compilatore, perché i messaggi e gli stub di servizio sono prodotti da generatori diversi:

```
protoc --java_out=src/main/java sensor.proto

protoc --plugin=protoc-gen-grpc-java=/percorso/protoc-gen-grpc-java-1.73.0 \
  --grpc-java_out=src/main/java sensor.proto
```

Artefatto	Prodotto da	Contenuto
Classi messaggio	--java_out	Reading, Ack, ... immutabili, con Builder annidato
SensorHubGrpc	--grpc-java_out	SensorHubImplBase (skeleton) e i tre stub (proxy)

Versioni del corso: `protobuf-java 4.31.1` (quindi `protoc 31.x`) e `grpc-java 1.73.0`. Un `protoc` più recente del runtime fa fallire il controllo `validateProtobufGencodeVersion` all'avvio, non in compilazione.

6. Domande di verifica concettuale

1. Il metodo di servizio è dichiarato `void` e non restituisce la risposta con `return`: quale pattern sostituisce il valore di ritorno, e perché questa forma è necessaria perché la stessa firma regga sia una risposta singola sia una risposta in streaming?
2. Il costruttore del cliente accetta un `Channel` e non un `ManagedChannel`. Che cosa dichiara, con questa scelta, la classe cliente riguardo alle proprie responsabilità, e quali due vantaggi pratici ne derivano?
3. Avviando il server con `forPort(0)`, su quale porta si mette in ascolto, e come fa il cliente a saperlo?
4. `start()` non è bloccante: che cosa succede al servente se dimentichi `awaitTermination()` nel `main`?
5. Quale corrispondenza c'è fra `SensorHubImplBase` + stub generati e lo schema proxy-skeleton realizzato a mano nella lezione precedente? Che cosa, in particolare, nasconde il codice generato?
6. Lo stesso `sensor.proto` compilato con il generatore Python produce stub interoperabili con questo servente Java. Quale elemento del file deve necessariamente coincidere perché l'invocazione vada a buon fine?

7. Errori tipici da evitare

- Dimenticare `onCompleted()` sul response observer: il cliente resta in attesa indefinita.
- Chiamare `onNext()` più di una volta in una RPC unaria: l'errore non è rilevato dal compilatore ma a runtime.
- Chiudere il canale prima che la chiamata asincrona sia terminata — da qui il latch.
- Aprire e chiudere un canale per ogni chiamata: il canale è costoso da creare (risoluzione del nome, TCP, handshake HTTP/2) ma è thread-safe e va riusato.
- Modificare una `HashMap` condivisa senza sincronizzazione, assumendo che il servente tratti una richiesta alla volta.
- Cambiare il numero di un campo dopo averlo definito: rompe la compatibilità sul wire, non la compilazione.

8. Checklist di autovalutazione

- ☐ Entrambe le invocazioni di `protoc` vanno a buon fine e le classi generate compilano.
- ☐ Una lettura non valida produce `INVALID_ARGUMENT` lato cliente e non abbatte il servente.
- ☐ Le letture in streaming compaiono *progressivamente*, non tutte insieme alla fine.
- ☐ Con lo stub asincrono il main prosegue durante i due secondi di `ComputeStats`.
- ☐ Tre clienti simultanei con un pool da due thread si comportano come ti aspettavi (annota cosa succede al terzo).
- ☐ Il programma termina da solo, senza thread appesi.

Estensione facoltativa

Compila lo stesso `sensor.proto` per Python e scrivi un piccolo cliente che invochi questo servente Java — è la verifica pratica della trasparenza di linguaggio, ed è esattamente la domanda che era rimasta aperta nei *Dubbi* della nota sulla lezione.

Quando hai una prima versione funzionante, mandami il codice: ti segnalo i punti critici senza riscriverlo.